# Unit-3
# SELCTING A MODEL

- Selecting a Model,
- Training a Model,
- Model Representation and Interpretability,
- Evaluating Performance of a Model, Improving Performance of a Model

# SOME IMPORTANT POINTS

- **Input variable** can be denoted by X, while individual input variables are represented as X1,X2,X3…..Xn and **output variable** by symbol Y.
- The relationship b/w X and Y is represented in general form.
  - **Y= f(X) + e**
  - **f is the target function** and **'e' is a random error term**.
- A **cost function (error function)** can tell how bad the model is performing and **loss function** a function defined on a data point, while cost function is for entire training data set.
- **Objective function** takes in data and model (along with parameters) as input and returns a value. Target is to find values of model parameter to maximize and minimize the return value.
- There is no one model that works best for every machine learning problem and that is what **'No Free Lunch'** theorem also states.
- **Supervised learning** for **solving predictive problems** and **unsupervised learning** which **solve descriptive problems**.

# Predictive Models

- Models for supervised learning or predictive models, as it understandable from name itself, **try to predict certain value in an input data set.**

- The learning model attempts to establish a relation b/w the target feature, i.e. **the feature being predicted**, and **predictor features.**

- The predictive models have a **clear focus on what they want to learn** and **how they want to learn**.

- Predictive models, in turn, **may need to predict the values of a category** or **class to which data instance belongs to**.

- Below are example of predictive…
  - **Predicting win/loss in a cricket match.**
  - **Predicting weather a transaction fraud.**
  - **Predicting whether a customer may move to another product.**

- The models which are **used for prediction of target features of categorical value are known as classification models.**

- **The target feature known as a class** and **the categories to which classes are divided into are called levels.**

- Some of the popular classification models include **k-Nearest Neighbor (kNN)**, **Naïve bayes**, and **Decision Tree**.

- Predictive models may also **be used to predict numerical values of the target feature based on the predictor features**. Some examples,
  - **Prediction of income growth in the succeeding year.**
  - **Prediction of rainfall amount in the coming monsoon.**

- The models which are used **for prediction of the numerical value** of the target feature of a data instance are **known as regression models.**
  - **Linear Regression**
  - **Logistic Regression**

# Descriptive Models

- Models for unsupervised learning or **descriptive models are used to describe a data set or gain insight from a data set.**

- There is **no target feature or single feature** of interest in case of unsupervised learning.

- **Based on the value of all features, interesting patterns** or **insight are derived about the data set**.

- **Descriptive models which group together similar data instance**, i.e. data instance having a similar value of the different features are **called clustering models**.

- Examples of **clustering** include..
  - **Customer grouping or segmentation based on social, demographic, national, etc factors**
  - **Grouping of music based on different aspects like type, language, time-period etc.**
  - **Grouping of commodities in an inventory.**

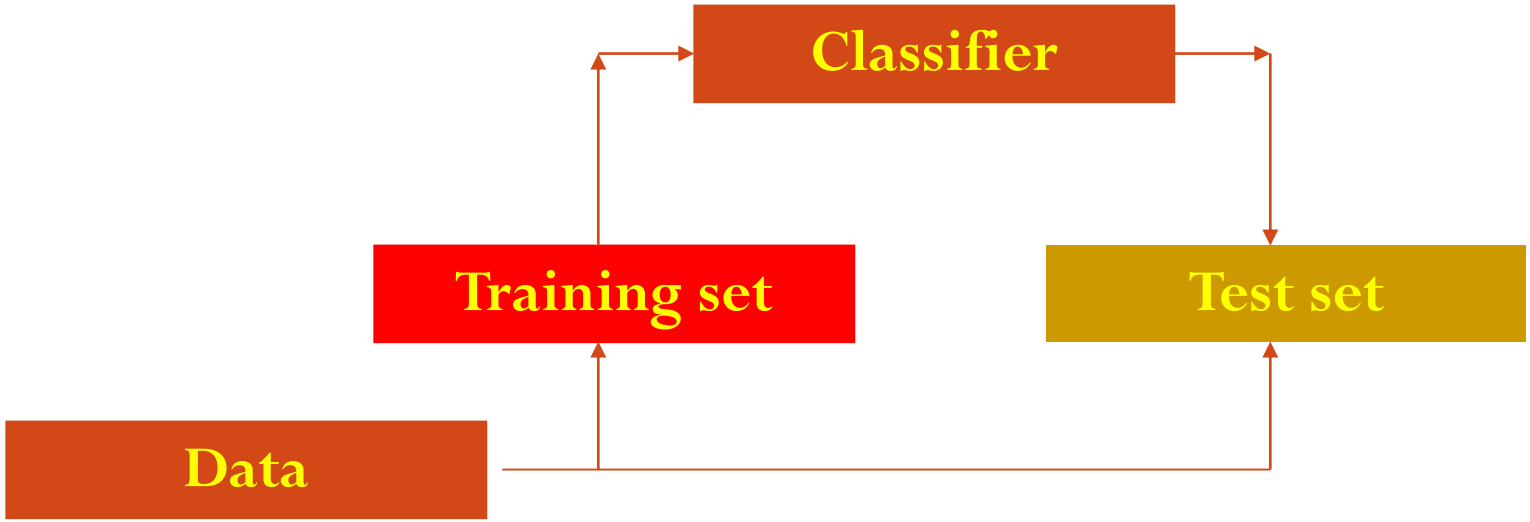
- The most popular **model for clustering is k-Means.**

- Descriptive models are **related to pattern discovery is used for market basket analysis of transactional data.**

# Training A Model (For supervised Learning)

## Holdout Method

- The hold-out method **splits the data into training data and test data.**
- Typical ratios used for **splitting the data set include 60:40, 80:20** etc.
- Then we **build a classifier using** the train data and test it using the test data.




- The hold-out method is usually **used when we have thousands of instances, including several hundred instances from each class**.
- This method is only used **when we only have one model to evaluate**.

- Once evaluation is complete, ***all the data*** **can be used to build the final classifier.**

- Generally, **the larger the training data the better the classifier** (but returns smaller).

- **The larger the test data the more accurate the error estimate.**

- The accuracy we receive from the validation set is not considered final and another hold-out dataset which is the test dataset is used to evaluate the final selected model and the error found here is considered as the generalization error.

Feature

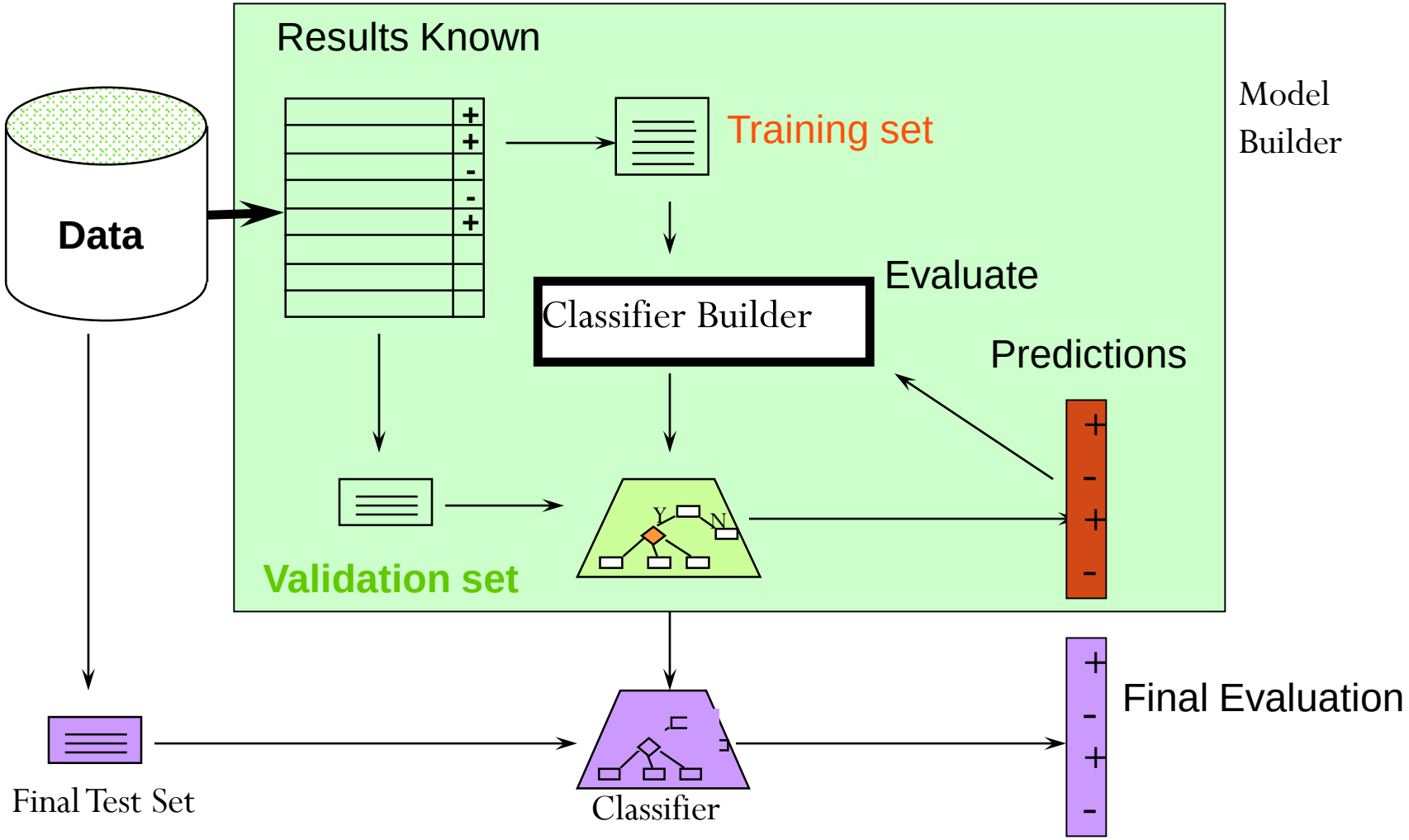
Instance

| x | y | z | class |
|---|---|---|---|
| 0.5351795492 | 0.9443102776 | 0.1582435145 | 1 |
| 0.2372136153 | 0.6406416746 | 0.2375491596 | 1 |
| 0.9115356348 | 0.3311024322 | 0.5615073269 | 0 |
| 0.5634070287 | 0.4183148035 | 0.151904445 | 0 |
| 0.3728975195 | 0.3816657621 | 0.616341473 | 1 |
| 0.6783527289 | 0.938524515 | 0.5269012505 | 1 |
| 0.09568660734 | 0.04465749689 | 0.0133451798 | 0 |
| 0.2173318229 | 0.6170559076 | 0.3122273853 | 1 |
| 0.818890594 | 0.7459451367 | 0.9026713492 | 0 |
| 0.6064854042 | 0.5945985792 | 0.2188024961 | 0 |
| 0.1546966824 | 0.1579937453 | 0.1333579164 | 0 |

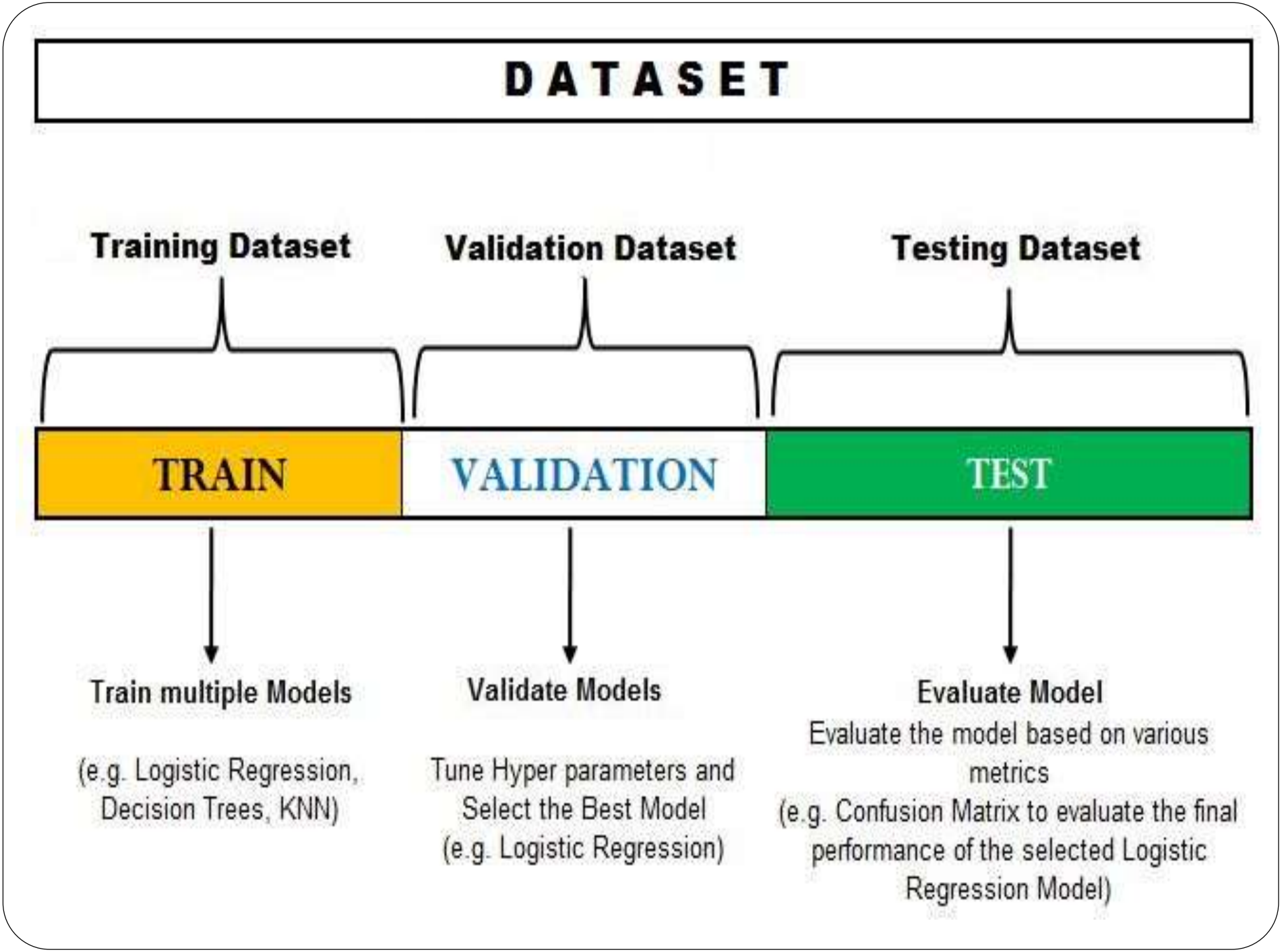
Train Dataset

Test Dataset

# Classification: Train, Validation, Test Split




**The test data can't be used for parameter tuning!**


DATASET
Training Dataset
Validation Dataset
Testing Dataset
TRAIN
VALIDATION
TEST
Train multiple Models
(e.g. Logistic Regression, Decision Trees, KNN)
Validate Models
Tune Hyper parameters and Select the Best Model (e.g. Logistic Regression)
Evaluate Model
Evaluate the model based on various metrics (e.g. Confusion Matrix to evaluate the final performance of the selected Logistic Regression Model)

# What is Cross Validation?

- **Cross Validation** is a very useful **technique for assessing the performance of machine learning models**.
- It **helps in knowing how the machine learning model would generalize to an independent data set.**
- You want to **use this technique to estimate how accurate the predictions your model will give in practice.**
- When you are given a machine learning problem, you will be given two type of data sets — known data (**training data set**) and unknown data (**test data set**).
- By using cross validation, you would be "testing" your machine learning model in the "training" phase to check for overfitting and to get an idea about how your machine learning model will generalize to independent data (he data set which was not used for training the machine learning model), which is the test data set given in the problem.

# K-fold Cross-validation method

- Usually, we split the data set into training and testing sets and use the training set to train the model and testing set to test the model.

- We then evaluate the model performance based on an error metric to determine the accuracy of the model.

- This method however, is not very reliable as the accuracy obtained for one test set can be very different to the accuracy obtained for a different test set.

- ***K-fold Cross Validation(CV)*** provides a solution to this problem by **dividing the data into folds** and **ensuring that each fold is used as a testing set at some point.**

- K-Fold CV is **where a given data set is split into a *K* number of sections/folds where each fold is used as a testing set at some point.**

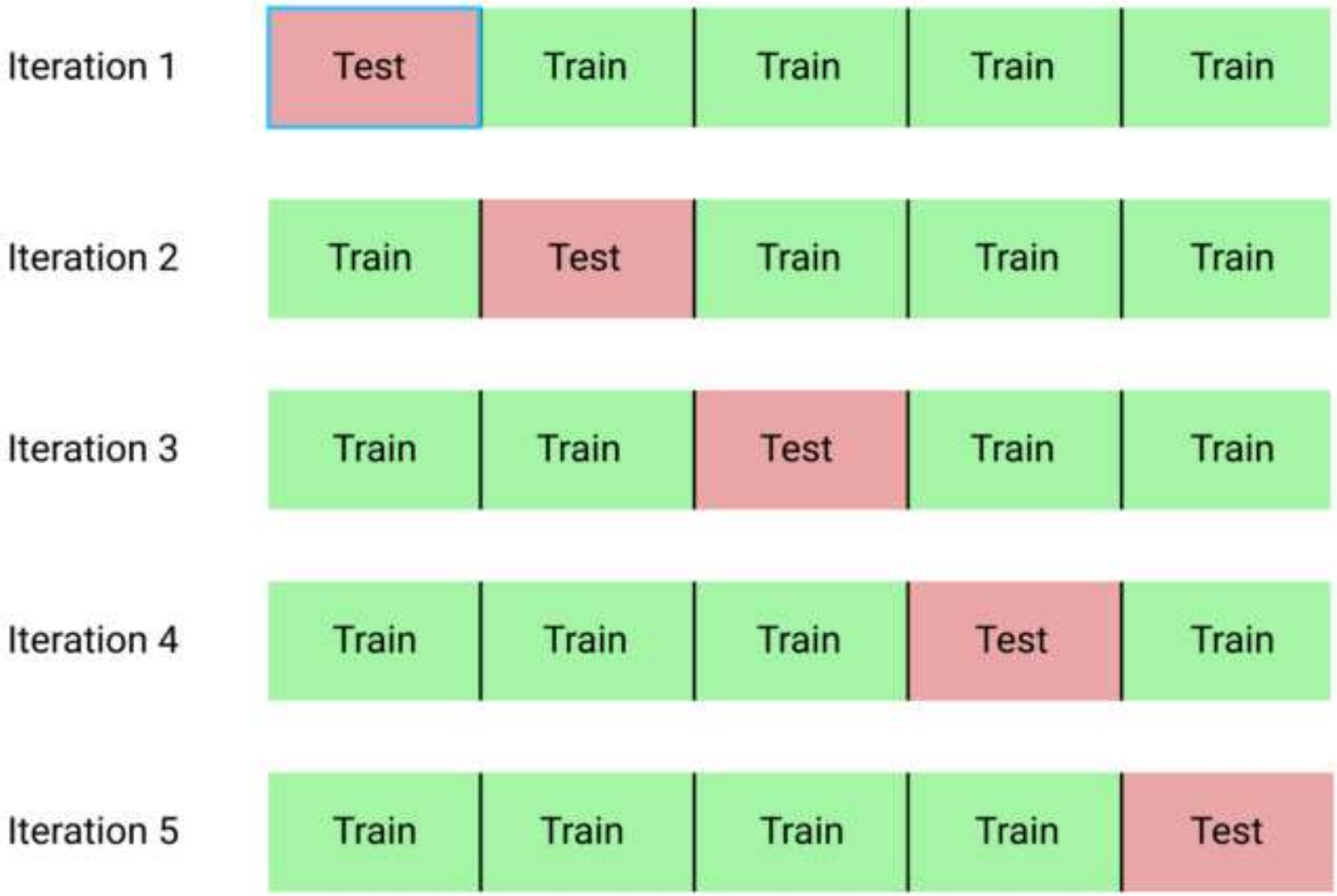
- This is one among the **best approach if we have a limited input data**.

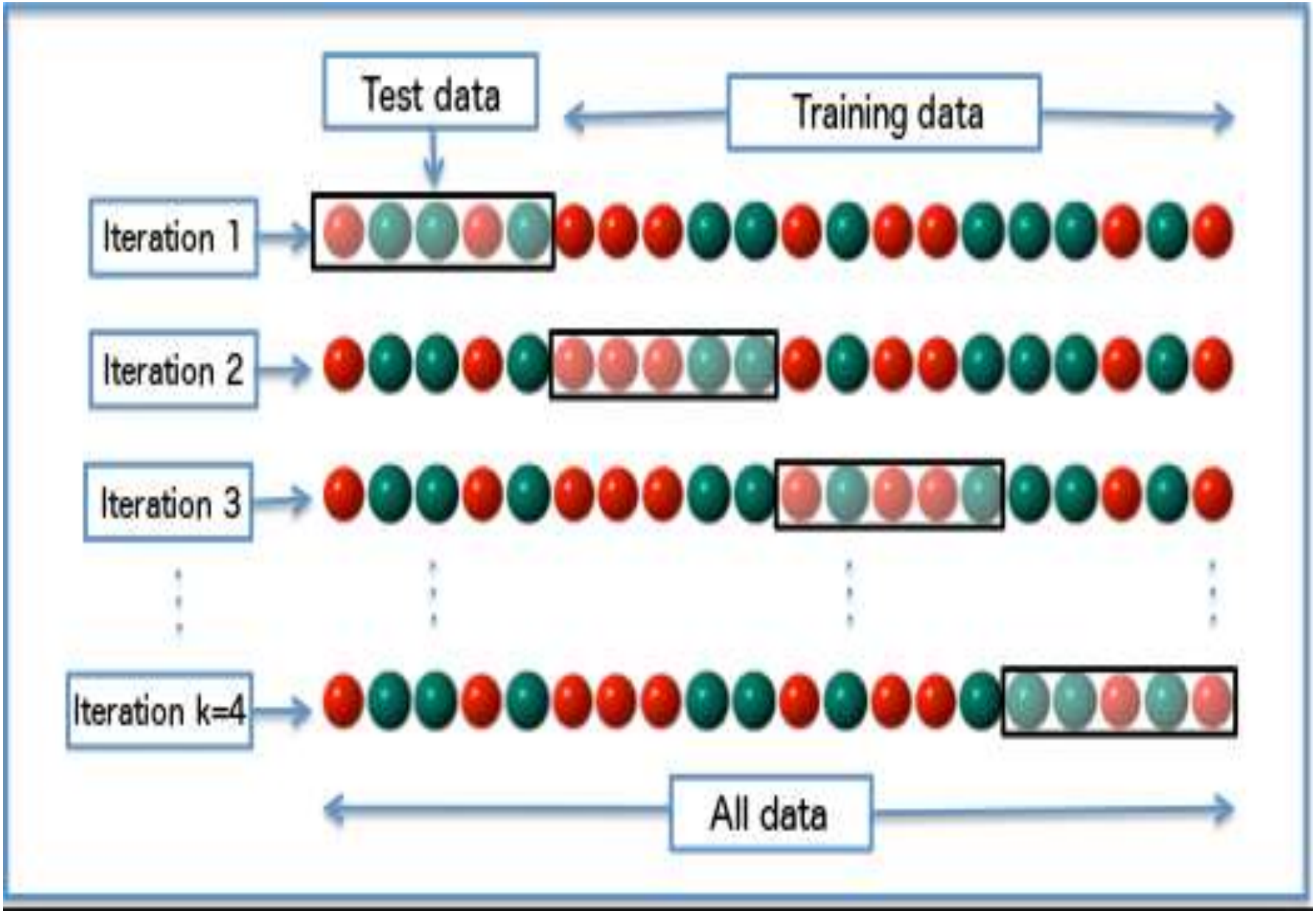
- **Lets take the scenario of 5-Fold cross validation(K=5).**
- Here, **the data set is split into 5 folds**. In the first iteration, **the first fold is used to test the model** and **the rest are used to train the model.**

- In the second iteration, **2nd fold is used as the testing set while the rest serve as the training set.**

- This **process is repeated until each fold of the 5 folds have been used as the testing set.**

- **Then take the average of your recorded scores**. That will be the performance metric for the model.


Iteration 1
Test
Train
Train
Train
Train
Iteration 2
Train
Test
Train
Train
Train
Iteration 3
Train
Train
Test
Train
Train
Iteration 4
Train
Train
Train
Test
Train
Iteration 5
Train
Train
Train
Train
Test


Test data
Training data
Iteration 1
Iteration 2
Iteration 3
Iteration k=4
All data

# Steps to perform K-fold validation

- **Split the entire data randomly into k folds** (value of k shouldn't be too small or too high, ideally we choose 5 to 10 depending on the data size).
- The **higher value of K leads to less biased model**, where as the **lower value of K is similar to the train-test split** approach we saw before.
- Then fit **the model using the K — 1 (K minus 1) folds** and **validate the model using the remaining Kth fold.** Note down the scores/errors.
- **Repeat this process until every K-fold serve as the test set**.
- **Then take the average of your recorded scores**. **That will be the performance metric for the model.**

```
import numpy as np
from sklearn.cross_validation import KFold

x = [1,2,3,4,5,6,7,8,9,10,11,12]
kf = KFold(12, n_folds=3)

for train_index, test_index in kf:
    print (train_index, test_index)

Fold 1: [ 4 5 6 7 8 9 10 11] [0 1 2 3]

Fold 2: [ 0 1 2 3 8 9 10 11] [4 5 6 7]

Fold 3: [0 1 2 3 4 5 6 7] [ 8 9 10 11]
```

# Approaches in k-fold cross validation

- Two approaches used..
  - **10-fold cross-validation (10-fold CV)**
  - **Leave-one-out cross validation (LOOCV)**

# 10-fold cross-validation (10-fold CV)

- With this method we have one data set which **we divide randomly into 10 parts.**
- We **use 9 of those parts for training** and **reserve one tenth for testing**.
- **We repeat this procedure 10 times each time reserving a different tenth for testing.**
- **Calculate the average of all the k test errors and display the result.**

Step 1: Divide the data set into k folds, here k is 10.

Step 2: Use one fold for testing a model built on all other data parts.

Step 3: Repeat the model building and testing for each of the data folds.

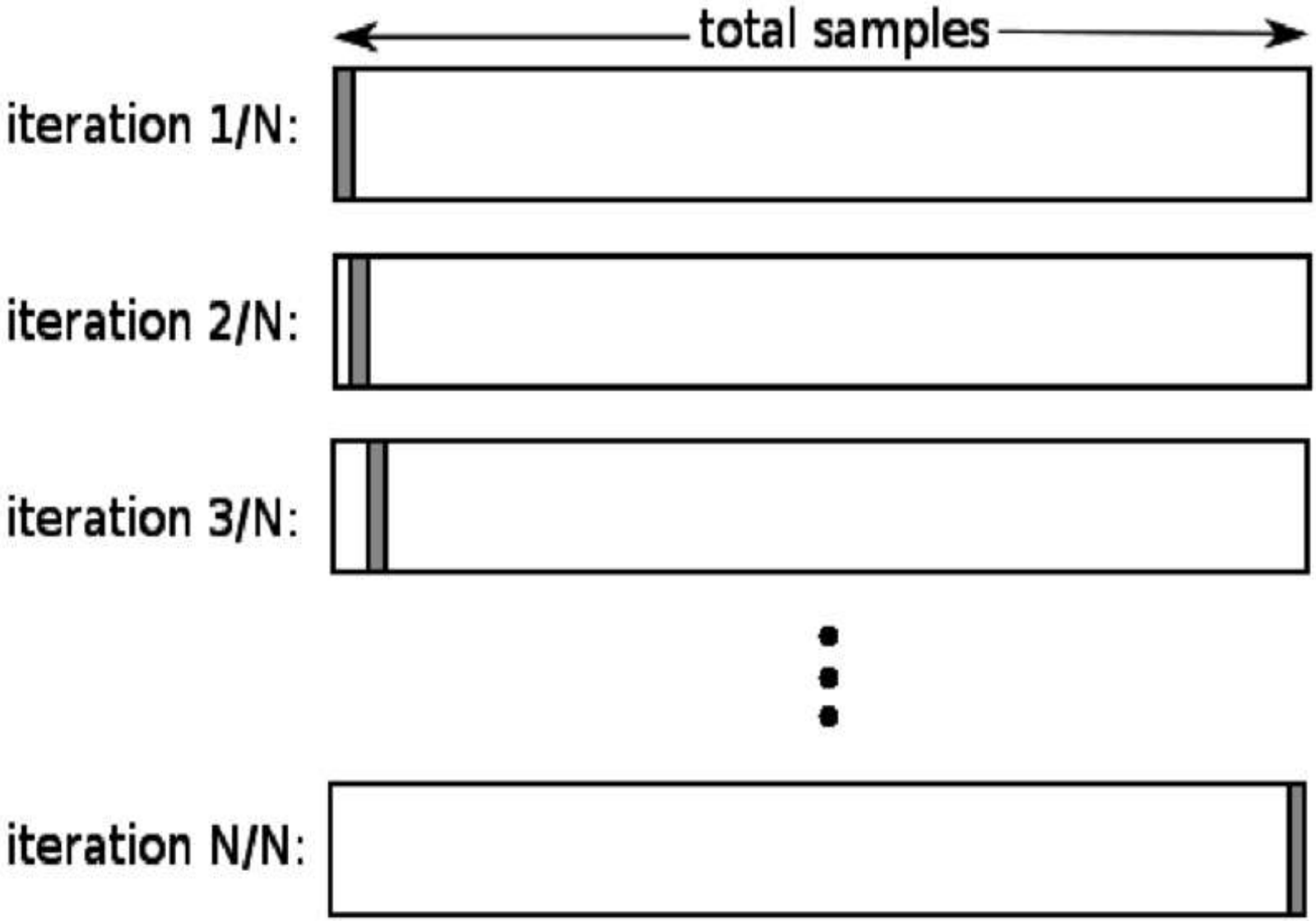
Step 4: Calculate the average of all of the k test errors and deliver this as result.

# Leave One Out Cross Validation (LOOCV)

- We can use LOOCV when data is limited and you want the absolute best error estimate for new data.
- LeaveOneOut (or LOO) is a simple cross-validation. **Each learning set is created by taking all the samples except one**, the test set being the sample left out.
- Thus, for n samples, we have n different training sets and n different tests set. This cross-validation procedure does not waste much data as **only one sample is removed from the training set.**
- This approach leaves 1 data point out of training data, i.e. **if there are n data points in the original sample then, n-1 samples are used to train the model** and **p points are used as the validation set.**

- This is repeated for all combinations in which the original sample can be separated this way, and **then the error is averaged for all** trials, to give overall effectiveness.

- The number of possible combinations is equal to the number of data points in the original sample or n.


total samples
iteration 1/N:
iteration 2/N:
iteration 3/N:
iteration N/N:

```
>>> from sklearn.model_selection import LeaveOneOut
>>> X = [1, 2, 3, 4]
 >>> loo = LeaveOneOut()
>>> for train, test in loo.split(X):
            . print("%s %s" % (train, test))

[1 2 3] [0]
[0 2 3] [1]
[0 1 3] [2]
[0 1 2] [3]
```

# Bootstrap Sampling

- **Bootstrap sampling or simply bootstrapping** is a popular way **to identify training and test data from input data set.**

- It uses the **technique of Simple Random Sampling with Replacement (SRSWR)** , which is a well **known technique in sampling theory for drawing random samples.**

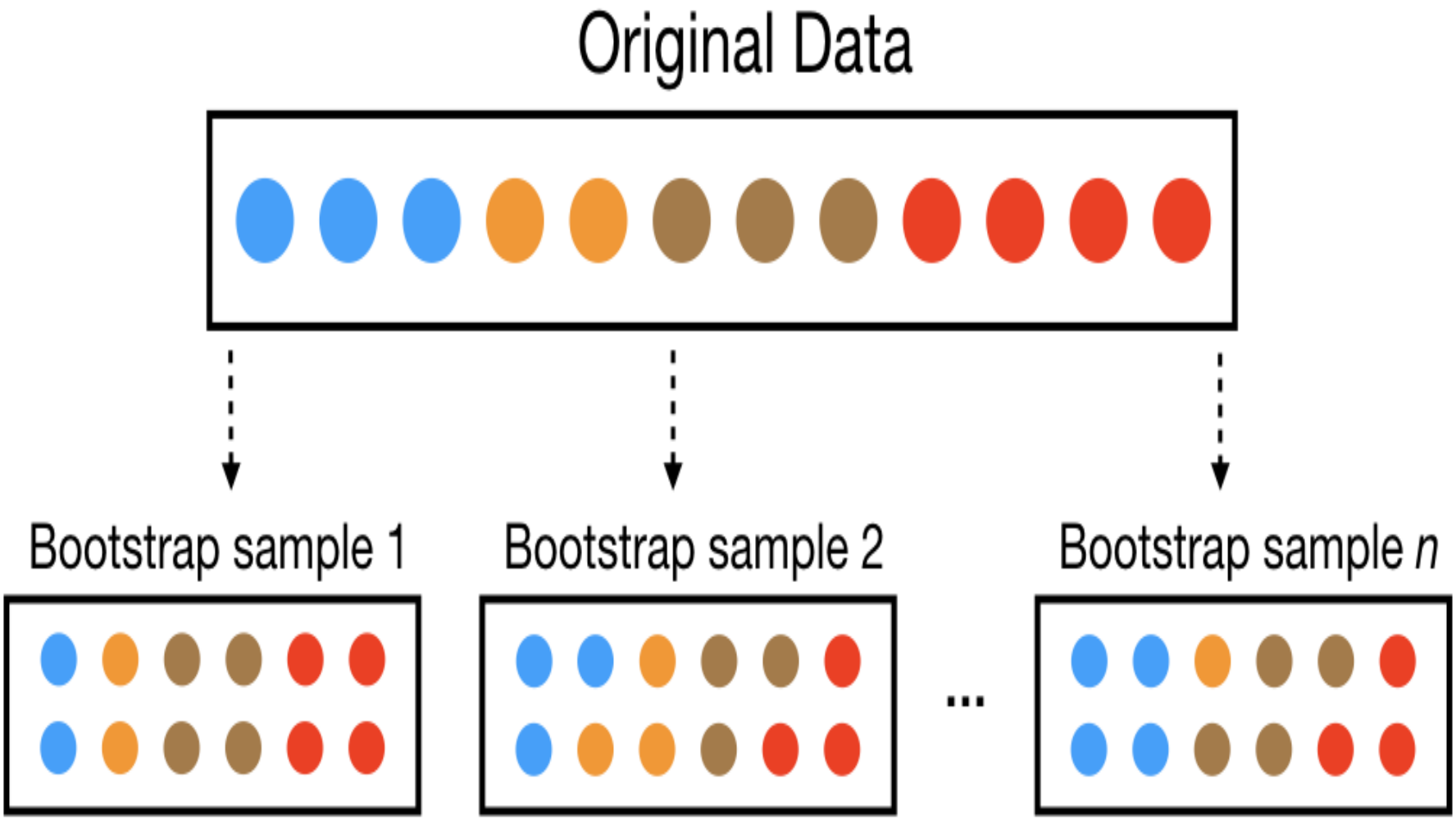
- **We have seen earlier that k-fold cross-validation** divides the data into separate partitions- say 10 partitions in case of 10 fold cross-validation. then it uses data instances from partitions as test data and remaining partitions as training data.

- Bootstrapping **randomly picks data instances from input data set**, with **the possibility of the same data instance to be picked multiple times.**

- This means that from the input data set having 'n' data instances, bootstrapping can create one or more training data sets having 'n' data instance, some of the data instances being repeated multiple times.

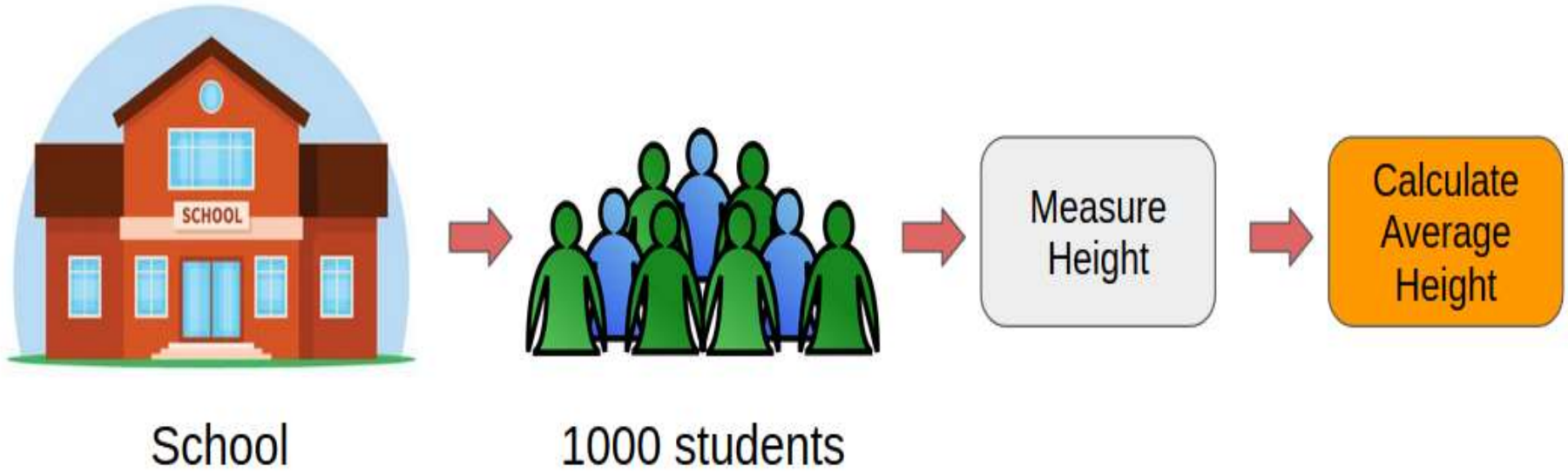
- **This technique is particularly useful in case of input data sets of small size**. i.e. having very less number of data instances.

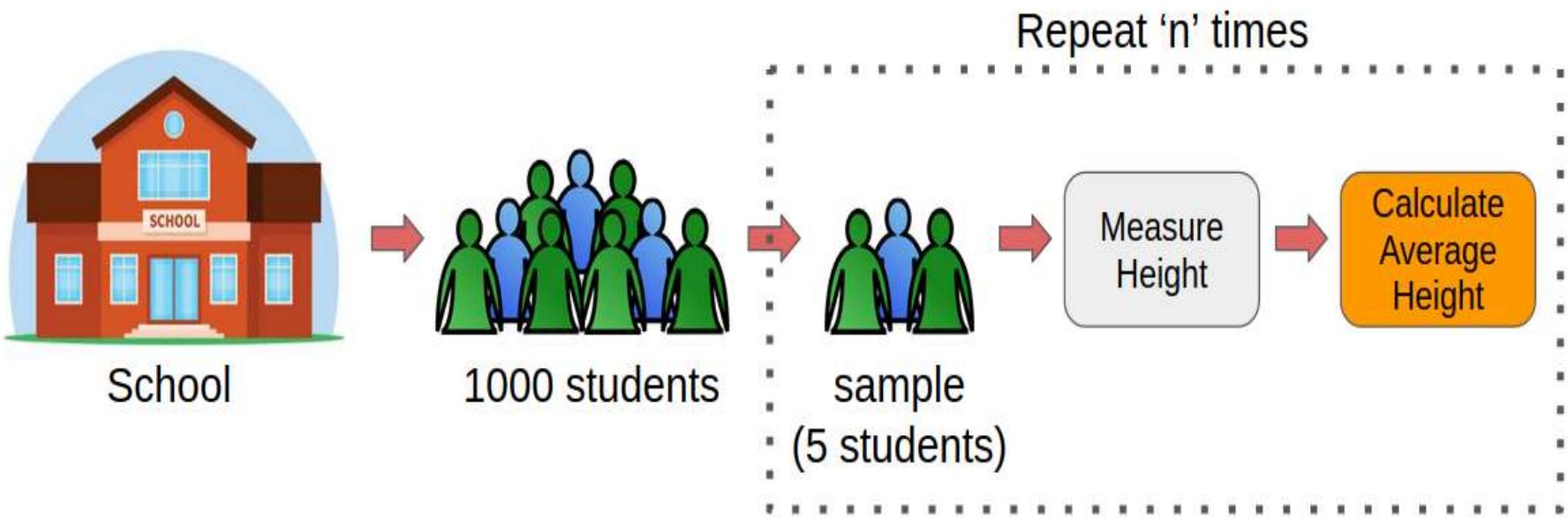
# Bootstrap Sampling



# Example of Bootstrap sampling

- Let's say we want to find the mean height of all the students in a school (which has a total population of 1,000). So, how can we perform this task?
- One approach is to measure the height of all the students and then compute the mean height.






- Instead of measuring the heights of all the students, **we can draw a random sample of 5 students and measure their heights.**
- **We would repeat this process 20 times and then average the collected height data of 100 students (5 x 20).**
- This average height would be an estimate of the mean height of all the students of the school.
- This is the basic idea of Bootstrap Sampling.

# Code for bootstrap sampling

```
# scikit-learn bootstrap
from sklearn.utils import resample

# data sample
data = [0.1, 0.2, 0.3, 0.4, 0.5, 0.6]

# prepare bootstrap sample
boot = resample(data, replace=True, n_samples=4, random_state=1)
print('Bootstrap Sample: %s' % boot)

# out of bag observations
oob = [x for x in data if x not in boot]
print('OOB Sample: %s' % oob)
```

**Output:**

Bootstrap Sample: [0.6, 0.4, 0.5, 0.1]

OOB Sample: [0.2, 0.3]

| Cross-Validation | Bootstrapping |
| --- | --- |
| It is a special variant of holdout method, called repeated holdout. Hence uses stratified random sampling approach (without replacement.) | It uses the technique of simple random sampling with replacement ( SRSWR), so the same data instance may be picked up multiple times in a sample. |
| The number of possible training/test data samples that can be drawn using this technique is finite. | Elements can be repeated in the sample, possible number of training/test data samples is unlimited. |

# Lazy vs. Eager Learning

- **Lazy learning** : Simply stores training data (or only minor processing) and waits until it is given a test tuple.

- Just store Data set **without** learning from it
- Start classifying data when it receive **Test data**
- So it takes less time learning and more time classifying data
- **e.g.** K - Nearest Neighbour, Case - Based Reasoning

- **Eager learning** : Given a set of training set, constructs a classification model before receiving new (e.g., test) data to classify.

- When it receive data set it starts classifying (learning)
- Then it does not wait for test data to learn.
- It's fast as it has pre-calculated algorithm.
- So it takes long time learning and less time classifying data.
- **e.g.** Decision Tree, Naive Bayes, Artificial Neural Networks

# Model Representation and Interpretability

- A model is said to be a **good machine learning model if it generalizes any new input data from the problem domain in a proper way.**

- This **helps us to make predictions in the future data, that data model has never seen.**

- Now, suppose we want to check how well our machine learning model learns and generalizes to the new data.

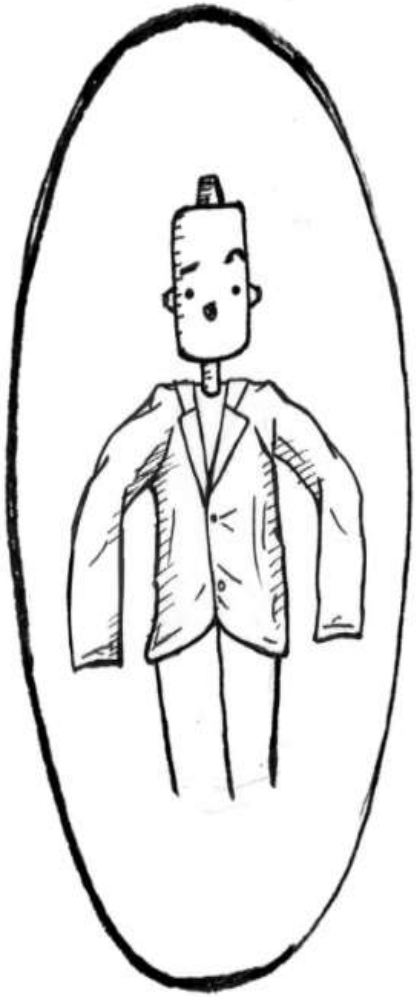
- For that we **have overfitting and underfitting**, which are majorly **responsible for the poor performances of the machine learning algorithms**.

- **Bias** – Assumptions made by a model to make a function easier to learn.
- **Variance** – If you train your data on training data and obtain a very low error, upon changing the data and then training the same previous model you experience high error, this is variance.

# MACHINE LEARNING GENERALIZATION

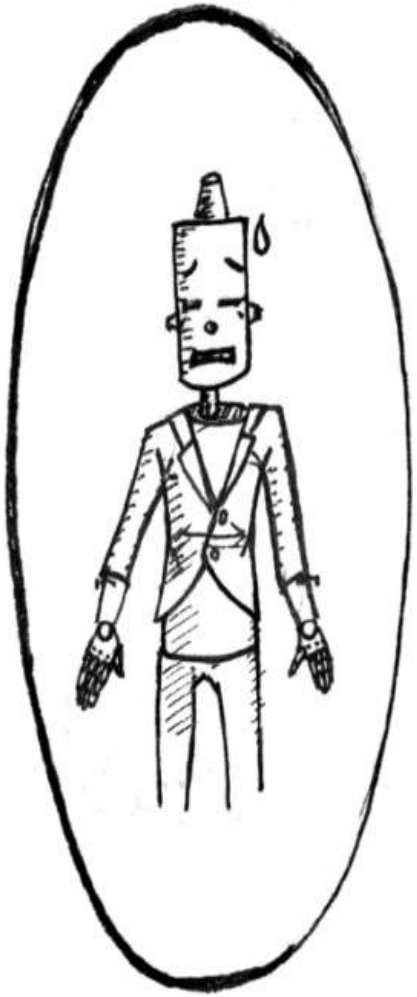
## FINDING THE PERFECT FIT

UNDERFIT



GOLDILOCKS ZONE

OVERFIT



# Underfitting

- A statistical model or a machine learning algorithm is said to have **underfitting when it cannot capture the underlying trend of the data.** *(It's just like trying to fit undersized cloths!)*
- **It only performs well on training data but performs poorly on testing data.**
- The input features are not explanatory enough to describe the target well.
- Underfitting **destroys the accuracy of our machine learning model**.
- Its occurrence simply **means that our model or the algorithm does not fit the data well enough.**
- It usually happens **when we have less data to build an accurate model and also when we try to build a linear model with a non-linear data.**
- In such cases the rules of the machine learning model are too easy and flexible to be applied on such minimal data and therefore the **model will probably make a lot of wrong predictions.**
- **Underfitting can be avoided by using more data and also reducing the features by feature selection.**

- **Underfitting – High bias and low variance.**
- **Techniques to reduce underfitting** :

1. Increase model complexity.
2. Increase number of features, performing feature engineering
3. Remove noise from the data.
4. Increase the number of epochs or increase the duration of training to get better results.

# Overfitting

- A statistical model is said to be overfitted, **when we train it with a lot of data** *(just like fitting ourselves in oversized cloths!)*.

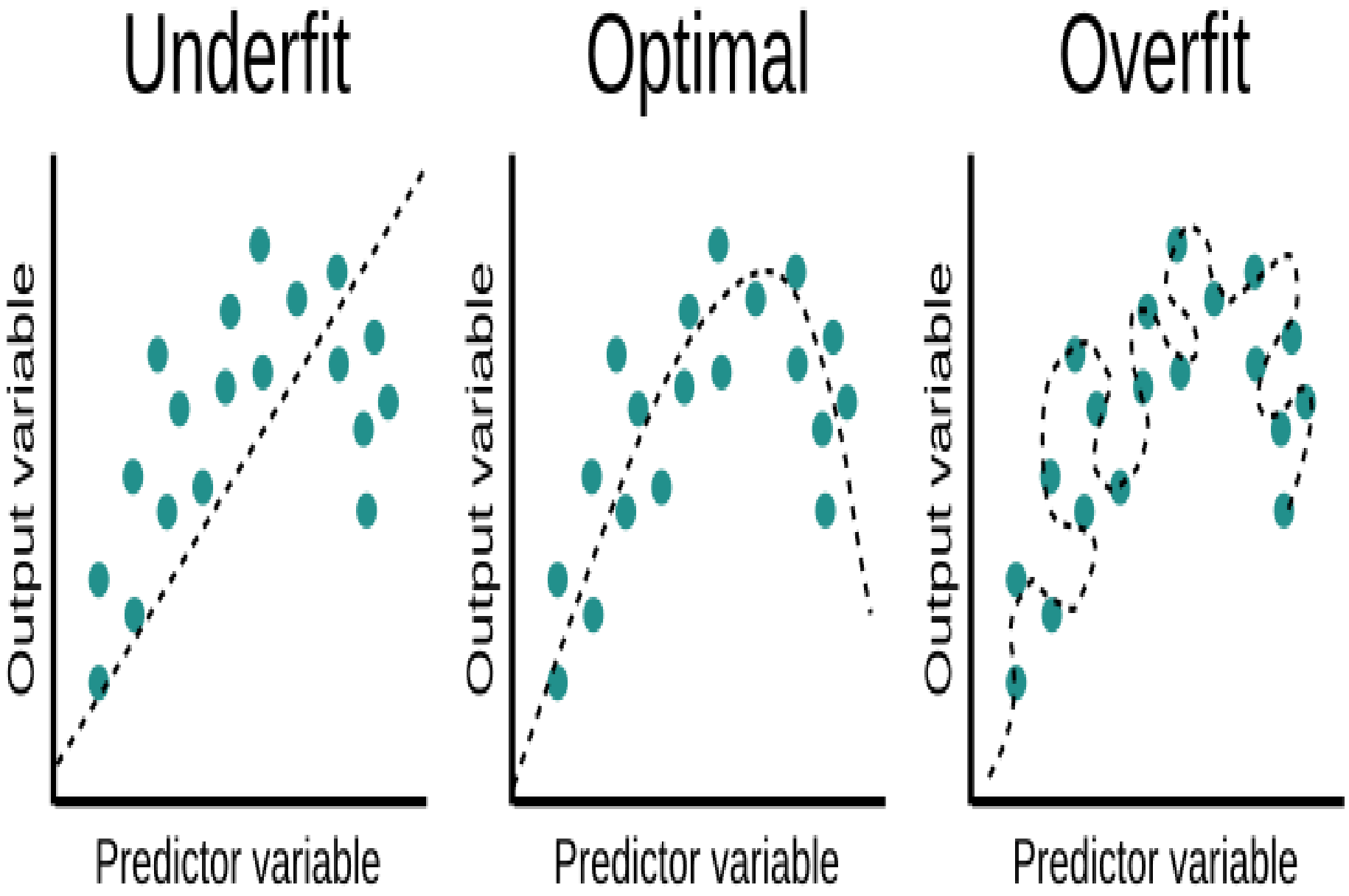
- The model does not make accurate predictions on testing data.
- When a model gets trained with so much data, it starts learning from the noise and inaccurate data entries in our data set. And when testing with test data results in High variance.
- Then the model does not categorize the data correctly, because of too many details and noise.

- The causes of **overfitting are the non-parametric and non-linear methods** because these types of machine learning algorithms have more freedom in building the model based on the dataset and therefore they can really build unrealistic models.

- A solution to **avoid overfitting is using a linear algorithm if we have linear data** or **using the parameters like the maximal depth if we are using decision trees.**

- **Overfitting – High variance and low bias.**
- **Techniques to reduce overfitting :**

1. Increase training data.
2. Reduce model complexity.
3. Early stopping during the training phase.
4. Use dropout for neural networks to tackle overfitting.


Underfit
Optimal
Overfit
Output variable
Output variable
Output variable
Predictor variable
Predictor variable
Predictor variable

# Bias-Variance Tradeoff (exchange)

- Whenever we discuss model prediction, **it's important to understand prediction errors (bias and variance).**
- There is a **tradeoff between a model's ability to minimize bias and variance.**

- If our **model is too simple and has very few parameters then it may have high bias and low variance (Underfitting).**
- On the other hand if our **model has large number of parameters then it's going to have high variance and low bias (overfitting).**

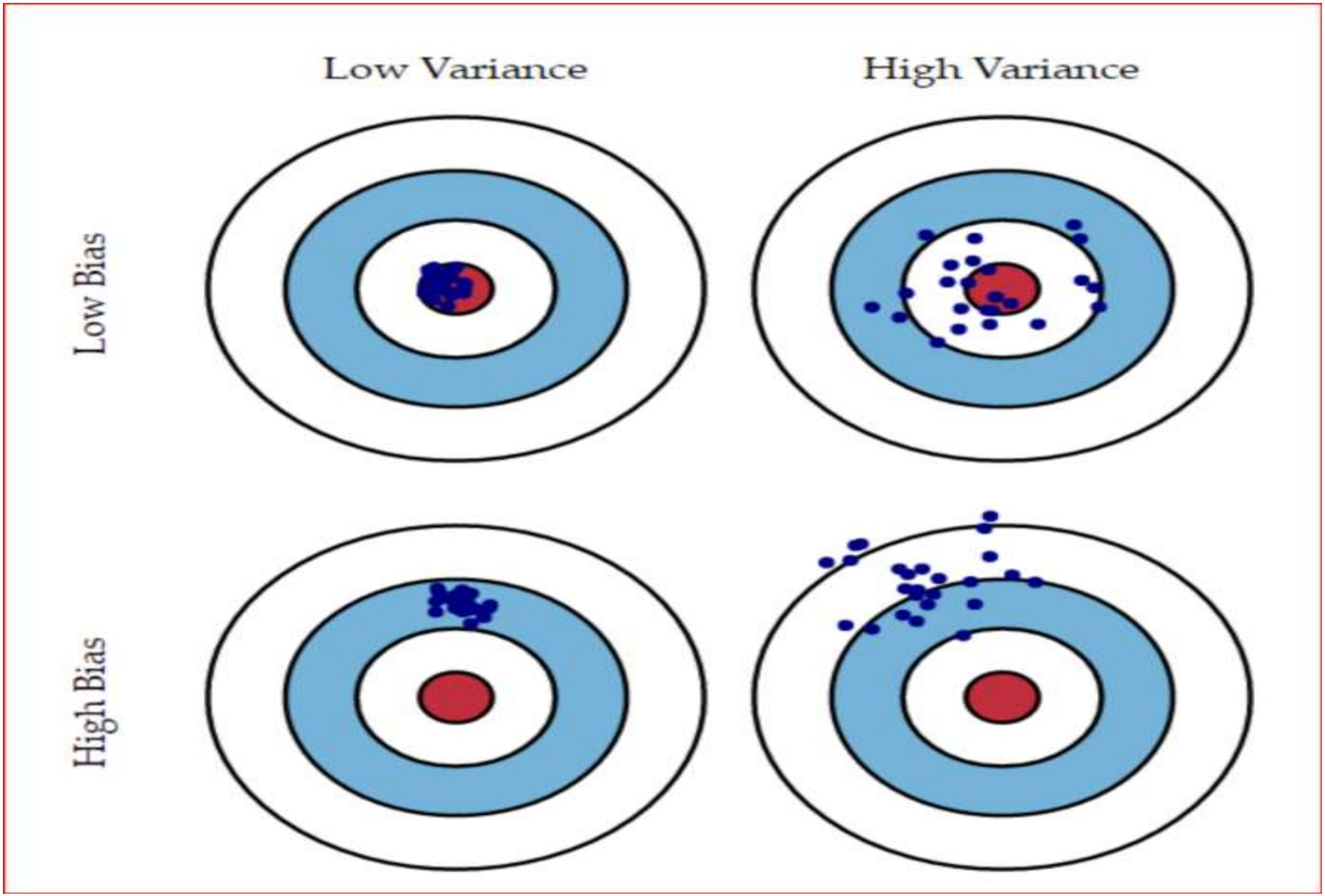
- So **we need to find the right/good balance without overfitting and underfitting the data.**

- This tradeoff in complexity is why there is a tradeoff between bias and variance.
- **An algorithm can't be more complex and less complex at the same time.**

- **Error due to Bias:** The error due to bias is taken as the difference between the expected (or average) prediction of our model and the correct value which we are trying to predict.
- Underfitting results in high bias.

- **Error due to Variance**: The error due to variance is taken as the variability of a model prediction for a given data point.
- Overfitting results in high variance.

# bias variance trade off



- **High Bias Low Variance**: Models are consistent but inaccurate on average.
- **High Bias High Variance** : Models are inaccurate and also inconsistent on average.
- **Low Bias Low Variance: Models are accurate and consistent on averages. We try hard for this in our model.**

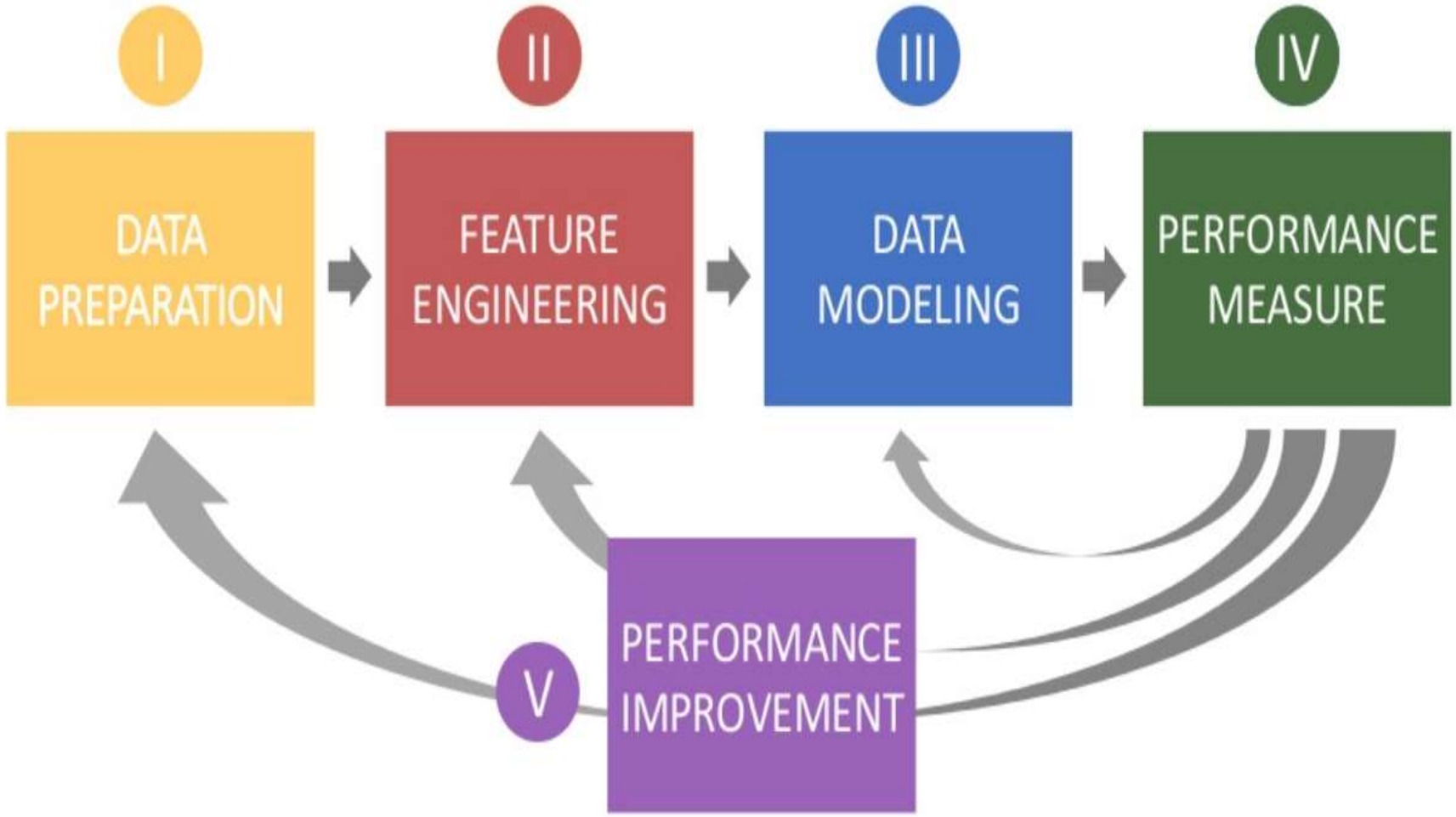
- **In fugure,** the best solution is to have a model with **low bias as well as low variance**. However, that may not be possible in reality.
- Hence, the goal of supervised ML is to achieve a balance b/w bias and variance.
- For ex., popular **supervised learning algorithm k-nearest Neighbors or kNN**, the user configurable parameter 'k' can be **used to do a trade off b/w bias and variance.**

# Evaluating Performance of a model

# WHAT IS A MACHINE LEARNING **MODEL**?

Clip slide

This is a **highly** iterative process, to repeat...




...until your model reaches a **satisfying performance!**

# Supervised learning - classification

- The responsibility of the **classification model is to assign class label to the target feature based on the value of the predictor feature**.
- **For ex.,** in the problem of predicting the win/loss in a cricket match, the classifier will assign a class value win/loss to target feature based on the values of other features like whether the team won the toss, number of spinners in the team, number of wins the tournament, etc.
- **To evaluate the performance of the model, the number of correct classifications or predictions made by the model has to be recorded.**
- A classification is said to be correct if, say for example **in the given problem, it has been predicted by the model that the team will win and it has actually win.**

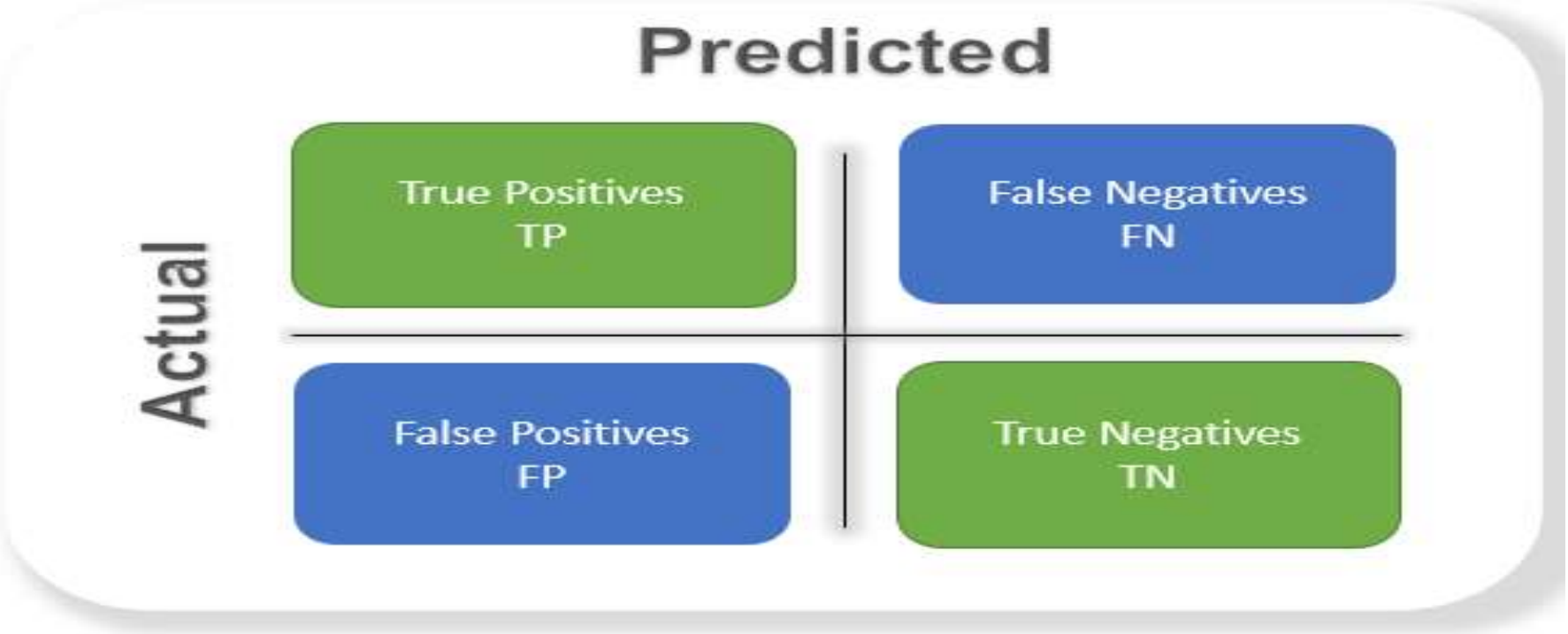
- Based on the **number of correct and incorrect classifications or predictions made by a model, the accuracy of the model is calculated.**
- There are 4 possibilities with regards to the cricket match win/loss prediction:
  - The model predicted win and the team won **(TP = True Positive)**
  - The model predicted win and the team lost **(FP = False Positive)**
  - The model predicted loss and the team won **(FN = False Negative)**
  - The model predicted loss and the team lost **(TN = True Negative)**

- **True positives (TP)**: Predicted positive and are actually positive.
- **False positives (FP)**: Predicted positive and are actually negative.
- **False negatives (FN)**: Predicted negative and are actually positive.
- **True negatives (TN)**: Predicted negative and are actually negative.

# Accuracy

- For any classification model, model accuracy is given by total number of correct classifications (either as the class of interest, i.e. True Positive or as not the class of interest, i.e. True Negative) divided by total number of classification done.

$$\text{Model accuracy} = \frac{TP + TN}{TP + FP + FN + TN}$$

# Confusion Matrix

- **A matrix containing correct and incorrect predictions in the form of TPs, FPs, FNs, and TNs is known as confusion matrix.**
- The win/loss predictions of cricket match has two classes of interest – win and loss.